

Linked List

It is a data type used for handling a group of logically related data items.

For example: record of student : name, roll number, marks

```
struct personal
{
    char    name[20];
    int     day;
    char    month[10];
    int     year;
    float   salary;
};
main()
{
    struct personal person;

    printf("Input Values\n");
    scanf("%s %d %s %d %f",
        person.name,
        &person.day,
        person.month,
        &person.year,
        &person.salary);
    printf("%s %d %s %d %f\n",
        person.name,
        person.day,
        person.month,
        person.year,
        person.salary);
}
```

```
#include <stdio.h>
struct employee
{
    int emp_id;
    char emp_name[20];
} e1;
int main()
{
    struct employee *e;
    printf("enter the details of employee1\n");

    scanf("%d%s",&e1.emp_id,e1.emp_name);
    printf("employees details\nemployee id
    =%d\nemployee name =
    %s",e1.emp_id,e1.emp_name);
    e=&e1;
    printf("\n%d%s",e->emp_id,e->emp_name);
    return 0;
}
```

Self Referential Structure

A self-referential structure is a structure that can have members which point to a structure variable of the same type

```
struct node  
{  
    int data;  
    struct node *next;  
};
```

- A linked list is a **data structure** which allows to store data dynamically and manage data efficiently.
- Linked List can be defined as collection of objects called **nodes** that are randomly stored in the memory.
- A node contains two fields i.e. data stored at that particular address and the pointer which contains the address of the next node in the memory.
- The last node of the list contains pointer to the null.
- There are three common types of Linked List.

1. Singly Linked List

2. Doubly Linked List

3. Circular Linked List

Linked List

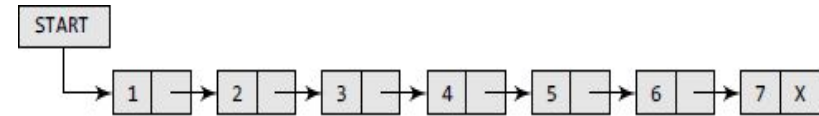


Figure 6.1 Simple linked list

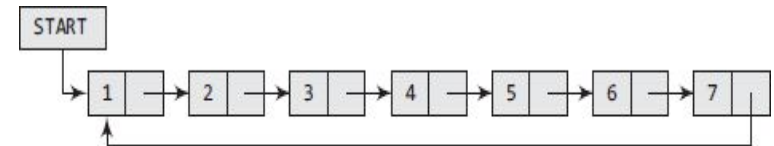


Figure 6.26 Circular linked list

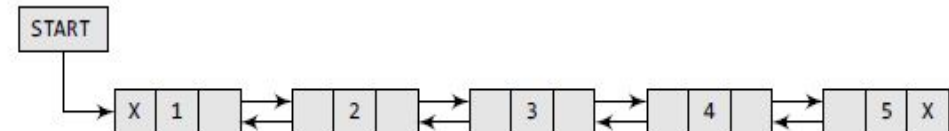


Figure 6.37 Doubly linked list

Linked list as Abstract data type

- Linked List is an Abstract Data Type (ADT) that holds a collection of Nodes, the nodes can be accessed in a sequential way. Linked List doesn't provide a random access to a Node.
- Usually, those Nodes are connected to the next node and/or with the previous one, this gives the linked effect. When the Nodes are connected with only the next pointer the list is called Singly Linked List and when it's connected by the next and previous the list is called Doubly Linked List.
- Perform following operations

Insertion at the beginning, end, at particular position

Deletion at the beginning, end, at particular position

Traversal

Reversal

Memory Representation of Linked List

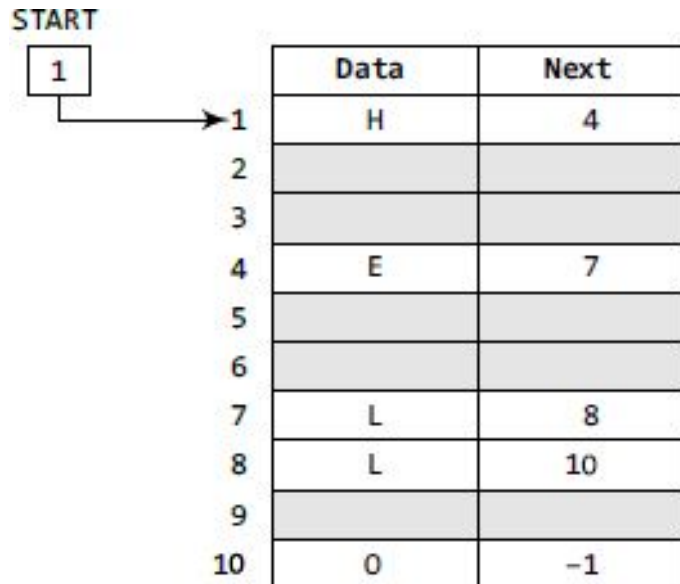


Figure 6.2 START pointing to the first element of the linked list in the memory

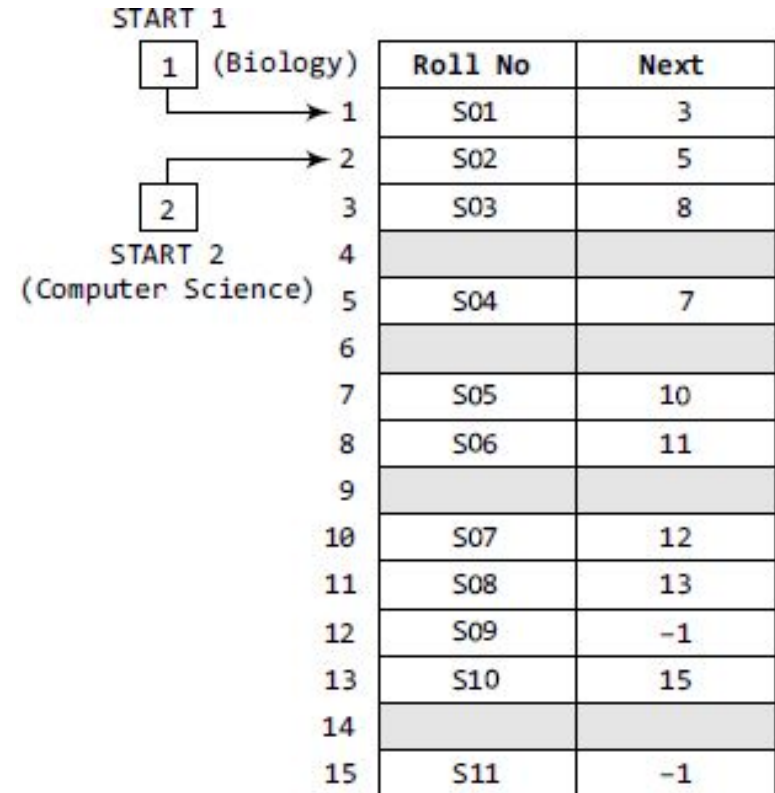


Figure 6.3 Two linked lists which are simultaneously maintained in the memory

Arrays Vs Linked List

ARRAY	LINKED LISTS
1. Arrays are stored in contiguous location.	1. Linked lists are not stored in contiguous location.
2. Fixed in size.	2. Dynamic in size.
3. Memory is allocated at compile time.	3. Memory is allocated at run time.
4. Uses less memory than linked lists.	4. Uses more memory because it stores both data and the address of next node.
5. Elements can be accessed easily.	5. Element accessing requires the traversal of whole linked list.
6. Insertion and deletion operation takes time.	6. Insertion and deletion operation is faster.

Array vs Linked List

Free Pool/ Avail List

list of available space is called the *free pool/ Avail list*

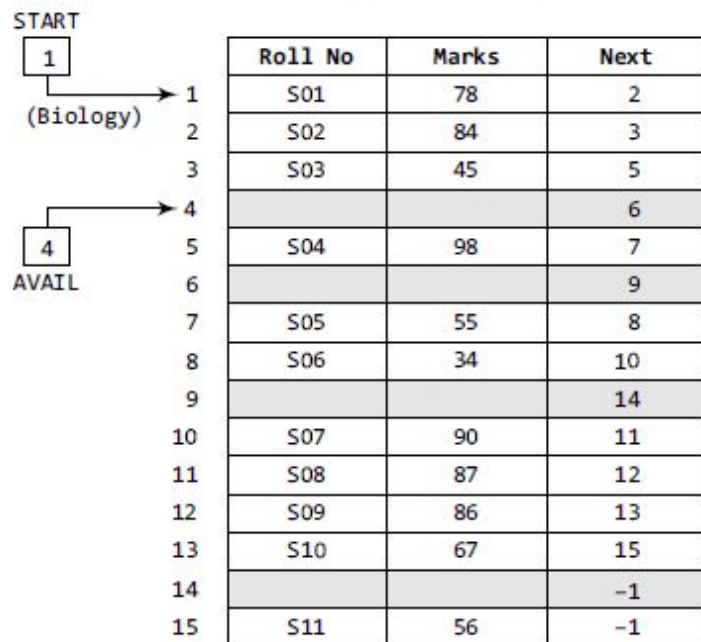
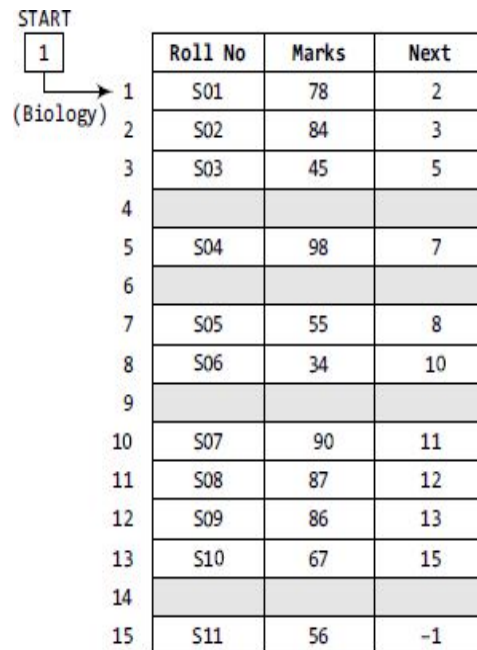
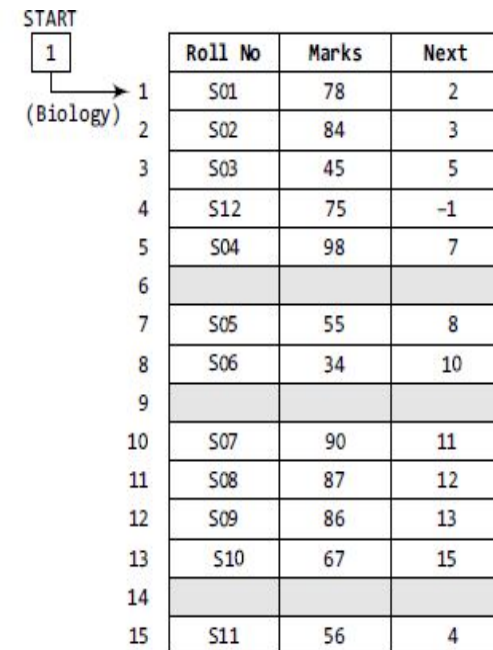


Figure 6.6 Linked list with AVAIL and START pointers



(a)



(b)

Figure 6.5 (a) Students' linked list and (b) linked list after the insertion of new student's record

Garbage Collection: The operating system scans through all the memory cells and marks those cells that are being used by some program. Then it collects all the cells which are not being used and adds their address to the free pool, so that these cells can be reused by other programs. This process is called *garbage collection*.

Singly Linked list

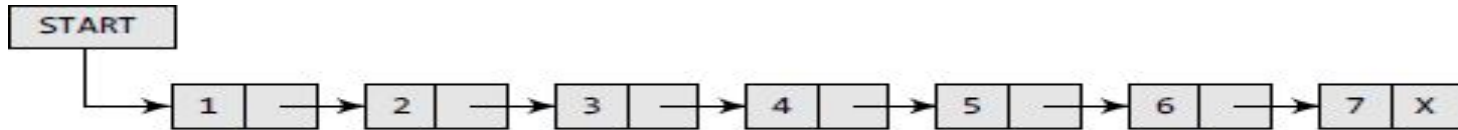
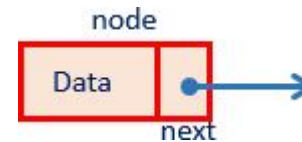


Figure 6.7 Singly linked list

- A start pointer stores address of the next node.
- Each node contains two parts information and link(pointer to next node)
- Link of the last node is NULL (indicates end of linked list).

```

struct node
{
    int data;
    struct node *next;
};
  
```



Note:

Such structures which contain a member field pointing to the same structure type are called **self-referential structures**.

Operations on Linked

- Traversing a list
 - Printing, finding minimum, etc.
- Insertion of a node into a list
 - At front, end and anywhere, etc.
- Deletion of a node from a list
 - At front, end and anywhere, etc.
- Comparing two linked lists
 - Similarity, intersection, etc.
- Merging two linked lists into a larger list
 - Union, concatenation, etc.
- Ordering a list
 - Reversing, sorting, etc.

Creation of Linked List

```
struct node  
{  
    int info;  
    struct node *link;  
} *start;
```

```
void create_List (int data)  
{  
    struct node *q, *tmp;  
    tmp = (struct node*) malloc (size of (struct node));  
    tmp->info = data;  
    tmp->link = NULL;  
    if (start == NULL):  
        start = tmp;  
}
```

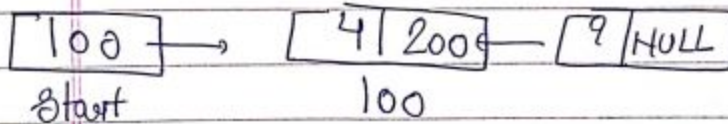
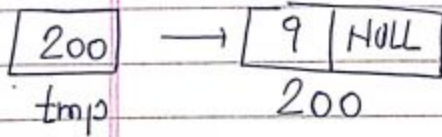
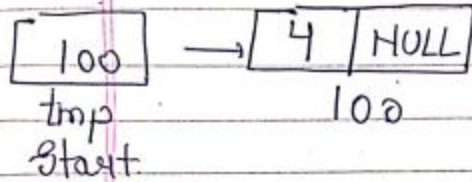
Creation of Linked List

```

else {
    q = start,
    while (q->link != NULL)
    {
        q = q->link;
    }
    q->link = tmp;
}
}

```

Time Complexity Create a list = $O(n)$



Traversal of Linked List

```

Void display()
{
    struct node * q;
    if (start == NULL)
    {
        printf("\n list is empty");
        return;
    }
    q = start;
    printf("\n list is");
    while (q != NULL)
    {
        printf("%d", q->info);
        q = q->link;
    }
    printf("\n");
}
Time Complexity of display = O(n)
    
```

Searching in a Linked

```

void Search (int data)
{
    struct node *ptr = start;
    int pos = 1;

    while (ptr != NULL)
    {
        if (ptr->info == data)
        {
            printf ("In Item is found at position %d, data, pos);
            return;
        }
        ptr = ptr->link;
        pos++;
    }
    if (ptr == NULL)
        printf ("In Item %d is not found", data);
}
    
```

Time Complexity Search = $O(n)$

Insertion in linked list

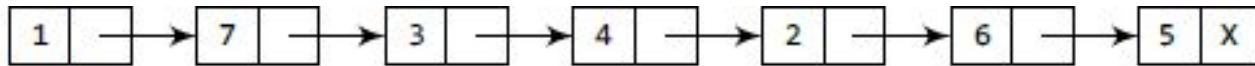
Case 1: The new node is inserted at the beginning.

Case 2: The new node is inserted at the end.

Case 3: The new node is inserted after a given node.

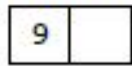
Overflow is a condition that occurs when no free memory cell is present in the system.

Insertion at the beginning

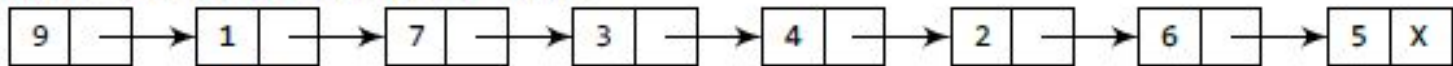


START

Allocate memory for the new node and initialize its DATA part to 9.

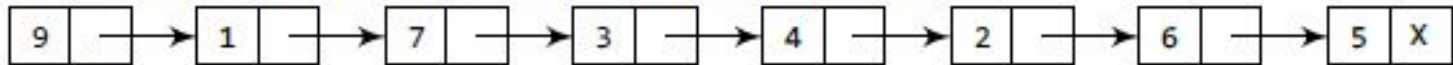


Add the new node as the first node of the list by making the NEXT part of the new node contain the address of START.



START

Now make START to point to the first node of the list.



START

Figure 6.12 Inserting an element at the beginning of a linked list

Insertion at the beginning

```
void Insert at Beg (int data)
```

```
{
```

```
    struct node *tmp;
```

```
    tmp = (struct node *) * malloc( size of (struct node))
```

```
    tmp → info = data;
```

```
    tmp → link = start;
```

```
    start = tmp;
```

```
}
```

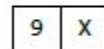
Time complexity insert at beg = $O(1)$

Insertion at the end of the linked list

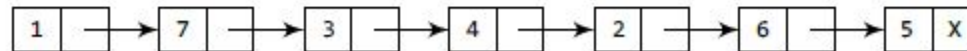


START

Allocate memory for the new node and initialize its DATA part to 9 and NEXT part to NULL.

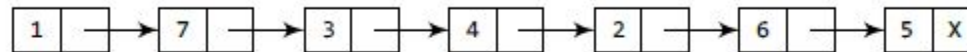


Take a pointer variable PTR which points to START.



START, PTR

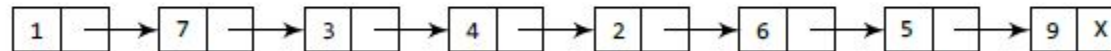
Move PTR so that it points to the last node of the list.



START

PTR

Add the new node after the node pointed by PTR. This is done by storing the address of the new node in the NEXT part of PTR.



START

PTR

Figure 6.14 Inserting an element at the end of a linked list

Insertion at the end of the linked list

Void insert at end (int data)

{

struct node * tmp, * q;

tmp = (struct node*) malloc (size of (struct node));

tmp → ~~link~~ info = data;

tmp → link = NULL;

q = start;

while (q → link != NULL)

{

q = q → link;

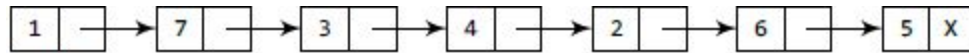
}

q → link = tmp;

}

Time Complexity Insert at end = $O(n)$

Insertion after a given node

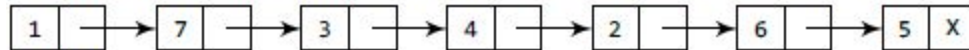


START

Allocate memory for the new node and initialize its DATA part to 9.



Take two pointer variables PTR and PREPTR and initialize them with START so that START, PTR, and PREPTR point to the first node of the list.

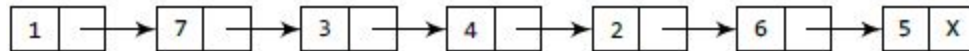


START

PTR

PREPTR

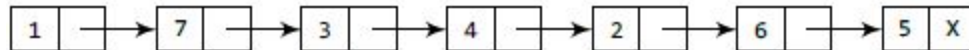
Move PTR and PREPTR until the DATA part of PREPTR = value of the node after which insertion has to be done. PREPTR will always point to the node just before PTR.



START

PREPTR

PTR

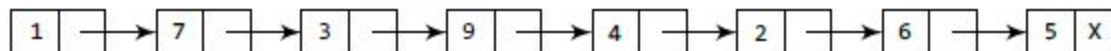
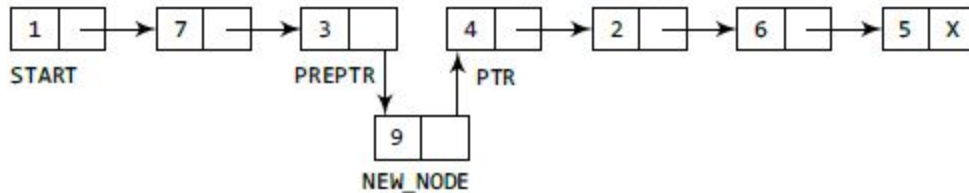


START

PREPTR

PTR

Add the new node in between the nodes pointed by PREPTR and PTR.



START

Figure 6.17 Inserting an element after a given node in a linked list

Insertion after a given node

After
Void insertAtPos (int data)
{
 struct node *tmp, *q;
 int i;
 q = start;
 for (i = 0; i < pos - 1; i++)
 {
 q = q->link;

if (q == NULL)
{
 printf ("Invalid position
 ~~There are less than i elements in the list~~
 ~~exit getch(); return;~~
}

tmp = (struct node *) malloc (size of (struct node));
tmp->info = data;
tmp->link = q->link;
q->link = tmp;
}

Time Complexity Insert at pos = $O(n)$

Deletion of a node from linked list

Case 1: The first node is deleted. Case 2: The last node is deleted.

Case 3: The node after a given node is deleted.

Underflow: when linked list is empty no deletion is possible such condition is known as underflow

Free Memory after deletion

- Do not forget to `free()` memory location dynamically allocated for a node **after deletion** of that node.
- It is the programmer's responsibility to free that memory block.
- Failure to do so may create a **dangling pointer** – a memory, that is not used either by the programmer or by the system.
- The content of a free memory is not erased until it is overwritten.

Deletion from first node

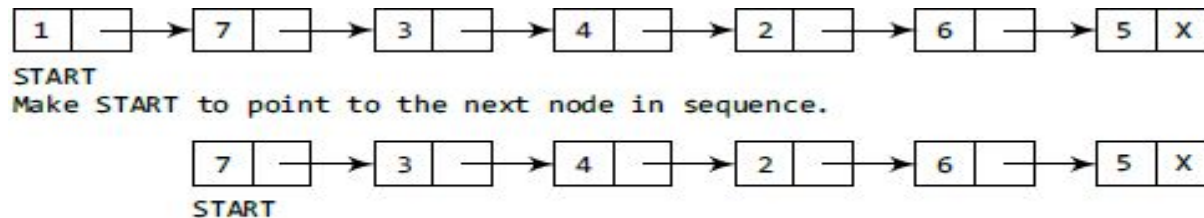


Figure 6.20 Deleting the first node of a linked list

```
void DeletefromBeg()
{
    struct node *tmp;
    tmp = start;
    if (start == NULL)
        printf("list is empty");
}
```

```
else
{
    tmp = start;
    start = start->next;
    free(tmp);
}
```

```

1
2 Void DeleteFromEnd()
3 {
4     struct node *prevnode, *tmp;
5     tmp = start;
6     while (tmp->next != NULL)
7     {
8         prevnode = tmp;
9         tmp = tmp->next;
10    }

```

```

11    if (tmp == start)

```

```

12        start = NULL;

```

```

13    else

```

```

14        prevnode->next = NULL;

```

```

15        free(tmp);

```

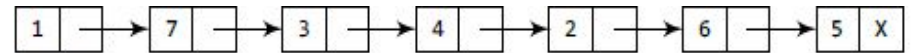
```

16    }

```

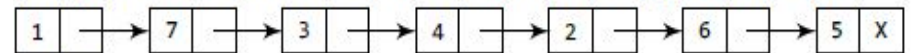
Time Complexity DeleteFromEnd = $O(n)$

Deletion of last node



START

Take pointer variables PTR and PREPTR which initially point to START.



START

PREPTR

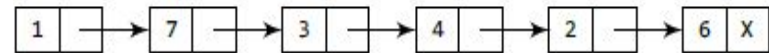
PTR

Move PTR and PREPTR such that NEXT part of PTR = NULL. PREPTR always points to the node just before the node pointed by PTR.



START

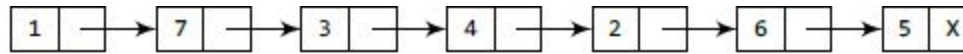
Set the NEXT part of PREPTR node to NULL.



START

Figure 6.22 Deleting the last node of a linked list

Delete a node after given node



START

Take pointer variables PTR and PREPTR which initially point to START.



START

PREPTR

PTR

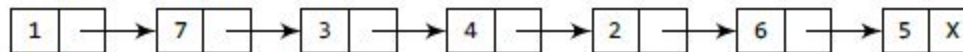
Move PREPTR and PTR such that PREPTR points to the node containing VAL and PTR points to the succeeding node.



START

PREPTR

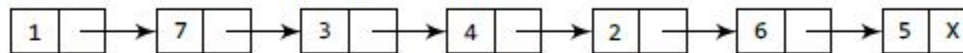
PTR



START

PREPTR

PTR



START

PREPTR

PTR

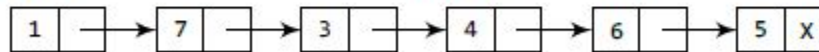
Set the NEXT part of PREPTR to the NEXT part of PTR.



START

PREPTR

PTR



START

Figure 6.24 Deleting the node after a given node in a linked list

Delete a node after given

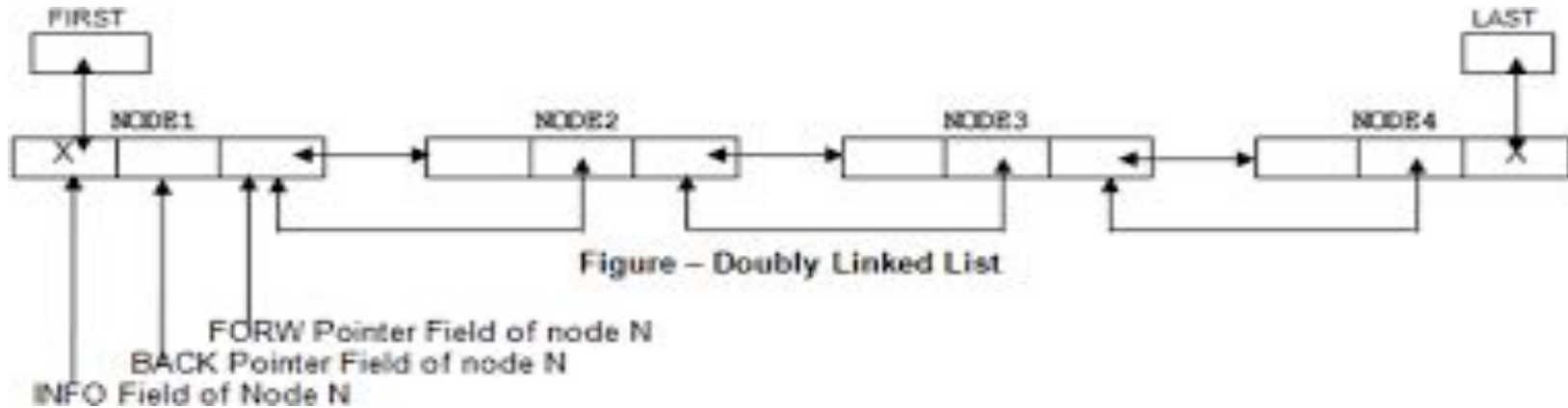
```

void deleteafterpos (int pos)
{
    struct node * ptr, * pto1;
    for (int i = 0; i < pos; i++)
    for (int i = 0; i < pos; i++)
    {
        pto1 = ptr;
        ptr = ptr->next;
        if (ptr == NULL)
        {
            printf ("Invalid position\n");
            return;
        }
        pto1->next = ptr->next;
        free(ptr);
    }
    printf ("Node Deleted\n", loc);
}

```

Time Complexity deletion after pos = $O(n)$

Doubly linked list



```
struct node
{
    struct node *prev;
    int data;
    struct node *next;
};
```


Doubly linked list VS singly linked list

Advantages over singly linked list

- 1) A DLL can be traversed in both forward and backward direction.
- 2) The delete operation in DLL is more efficient if pointer to the node to be deleted is given.

Disadvantages over singly linked list

- 1) Every node of DLL Require extra space for an previous pointer.
- 2) All operations require an extra pointer previous to be maintained.

Operations on Doubly linked list

Insertion at beginning

Insertion at end

Insertion after given node

Traversal

Deletion at the beginning

Deletion at the end

Delete a node

Creation of Doubly Linked List

```
struct node
```

```
{
    int info;
    struct node *prev;
    struct node *next;
} *start;
```

```
void create_list(int x)
```

```
{
    struct node *tmp, *q;
    tmp = (struct node *) malloc (size of (struct node));
    tmp->info = x;
    tmp->prev = null;
    tmp->next = null;
```

```
if (start == null)
```

```
{
    tmp->prev = null;
    start->prev = tmp;
    start = tmp;
}
```

```
else
{
    q = start;
```

```
while (q->next != NULL)
```

```
{
    q = q->next;
```

```
}
```

```
q->next = tmp;
```

```
tmp->prev = q;
```

```
}
```

Time complexity of create Double linked list = $O(n)$

Insertion at Beginning

```

void addatbeg(int x)
{
    struct node *tmp;
    tmp = (struct node *) malloc (size of (struct node));
    tmp->info = x;
    tmp->prev = null;
    tmp->next = null;

```

```

    if (start == null)

```

```

    {

```

```

        start = tmp;

```

```

    }

```

```

    else

```

```

    {

```

```

        tmp->next = start;

```

```

        start->prev = tmp;

```

```

        start = tmp;

```

```

    }

```

Time Complexity insert at beg of the ~~link~~ doubly linked list = $O(1)$

Insertion at End

```

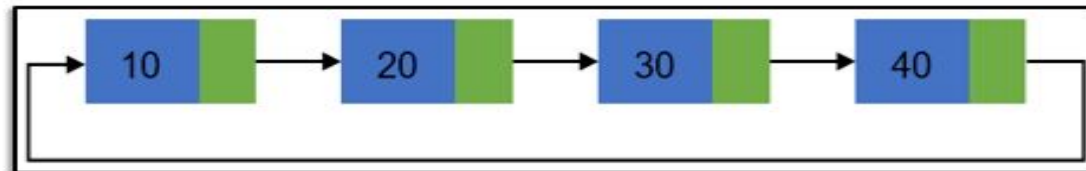
void insertatend (int x)
{
    struct node *tmp, *q;
    tmp = (struct node*) malloc (size of (struct node));
    tmp->info = x;
    tmp->prev = null;
    tmp->next = null;
    if (start == NULL)
        start = tmp;
    else
    {
        q = start;
        while (q->next != NULL)
        {
            q = q->next;
        }
        q->next = tmp;
        tmp->prev = q;
    }
}
    
```

Time Complexity insert at end = $O(n)$

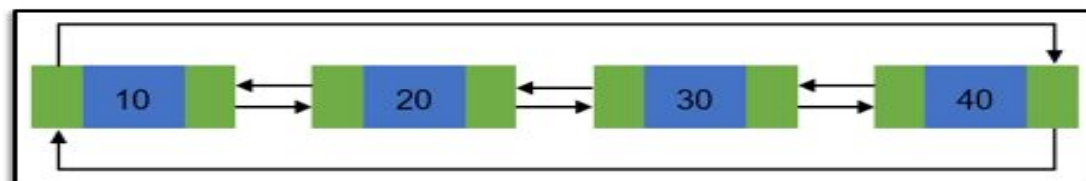
Circular Linked List

- The pointer from the last element in the list points back to the first element.
- A circular linked list is basically a linear linked list that may be **single-** or **double-linked**.
- The only difference is that **there is no any NULL** value terminating the list.
- In fact in the list every node points to the next node and last node points to the first node, thus forming a circle. Since it forms a **circle** with **no end to stop** it is called as **circular linked list**.
- In circular linked list there can be no starting or ending node, whole node can be **traversed from any node**.
- In order to traverse the circular linked list, only once we need to traverse entire list until the **starting node is not traversed again**.

Basic structure of singly circular linked list:



Doubly circular linked list:



Advantages and Disadvantages of Circular Linked List

Page No. _____

Advantages of a Circular linked list

- Entire list can be traversed from any node.
- Circular lists are the required data structure when we want a list to be accessed in a circle or loop.
- Despite of being singly circular linked list we can easily traverse to its previous node, which is not possible in singly linked list.

Disadvantages of Circular linked list

- Circular list are complex as compared to singly linked lists.
- Reversing of circular list is a complex as compared to singly or doubly lists.
- If not traversed carefully, then we could end up in an infinite loop.

Creation and Traversal of Circular Linked List

```
#include <stdio.h>
#include <stdlib.h>

/* Basic structure of Node */

struct node {
    int data;
    struct node * next;
} *head;

int main()
{
    int n, data;
    head = NULL;

    printf("Enter the total number of nodes in list: ");
    scanf("%d", &n);
    createList(n);           // function to create circular linked list
    displayList();           // function to display the list

    return 0;
}
```



```
void createList(int n)
{
    int i, data;
    struct node *prevNode, *newNode;
    if(n >= 1){ /* Creates and links the head node */
        head = (struct node *)malloc(sizeof(struct node));

        printf("Enter data of 1 node: ");
        scanf("%d", &data);

        head->data = data;
        head->next = NULL;
        prevNode = head;

        for(i=2; i<=n; i++){ /* Creates and links rest of the n-1 nodes */
            newNode = (struct node *)malloc(sizeof(struct node));

            printf("Enter data of %d node: ", i);
            scanf("%d", &data);

            newNode->data = data;
            newNode->next = NULL;
            prevNode->next = newNode; //Links the previous node with newly created node
            prevNode = newNode;      //Moves the previous node ahead
        }

        prevNode->next = head; //Links the last node with first node
        printf("\nCIRCULAR LINKED LIST CREATED SUCCESSFULLY\n");
    }
}
```

```
void displayList()
{
    struct node *current;
    int n = 1;

    if(head == NULL)
    {
        printf("List is empty.\n");
    }
    else
    {
        current = head;
        printf("DATA IN THE LIST:\n");

        do {
            printf("Data %d = %d\n", n, current->data);

            current = current->next;
            n++;
        }while(current != head);
    }
}
```



Vivekananda Institute
of Professional Studies

END